Question 1 (11 points)

Effectuez les conversions demandées à l'aide de la procédure appropriée. Donnez la trace de votre solution.

(a) $(264.906\,25)_{10}$ en base 2 :

264	2
132	0
66	0
33	0
16	1
8	0
4	0
2	0
1	0
0	1

2	0,906 25
1	,812 5
1	,625
1	,25
0	,5
1	,0

$\boxed{1\,0000\,1000.1110\,1_2}$

(b) $(2\,765.6)_{10}$ en base 8 :

2765	8
345	5
43	1
5	3
0	5

8	0,6
4	,8
6	,4
3	,2
1	,6
4	,8

$\boxed{5315,\overline{4631}_8}$

(c) $(5\,789.343\,75)_{10}$ en base 16 :

5 789	16
361	D
22	9
1	6
0	1

16	0,343 75
5	,5
8	,0

$\boxed{1\,69D.58_{16}}$

(d) Vérifiez votre calcul de c).

$169D.58_{16} = 1\times16^3 + 6\times16^2 + 9\times16^1 + 13\times16^0 + 5\times16^{-1} + 8\times16^{-2}$

$= 5\,789,343\,75_{16}$ ✓

Question 2 (6 points)

Convertissez en **décimal** les nombres suivants à l'aide de la procédure appropriée. Donnez la trace de votre solution.

(a) $(1101\,0110.101)_2$

$= 2 + 4 + 16 + 64 + 128 + \frac{1}{2} + \frac{1}{8}$

$= \boxed{214,625_{10}}$

(b) $(714.15)_8$

$= 7 \times 8^2 + 1 \times 8^1 + 4 \times 8^0 + 1/8 + 5/64$

$= \boxed{460.203125_{10}}$

(c) $(AB7.F1)_{16}$

$= 10 \times 16^2 + 11 \times 16^1 + 7 \times 16^0 + 15/16 + 1/256$

$= \boxed{2743.941406 25_{10}}$

← important de calculer séparement la partie fractionnaire pour avoir une réponse precise.

Question 3 (12 points)

Complétez les cases vides du tableau suivant. Donnez la trace de votre solution.

	Binaire	Octal	Hexadécimal
a)	010 1101 0100.1011 100	1324.56_8	$2D4.B8_{16}$
b)	111 110 101.100 1 1111 0101.1$_2$	765.4	$1F5.8_{16}$
c)	0010 1010 0100.1111 0001 10 1010 0100.1111 0001$_2$	1244.742_8	$2A4.F1$

Trace : 0001 1111 0101.1000

0010 1010 0100.1111 0001 0

Question 4 (16 points)

Effectuez les conversions demandées à l'aide de la procédure appropriée. Donnez la trace de votre solution.

(a) $(12\,345\,678.987)_{10}$ en base 2 :

c'est + rapide de passer par la base 16.

12 345 678	16
771 604	E
48 225	4
3 014	1
188	6
11	C
0	B

16	0,987
F	,792
C	,672
A	,752
⋮	⋮

$BC\,614E.FCA\ldots_{16} = 1011\ 1100\ 0110\ 0001\ 0100\ 1110.1111\ 1100\ 1010\ldots_2$

(b) $(BA0.BAB)_{16}$ en base 10 :

$BA0.BAB_{16} = 11\times16^2 + 10\times16^1 + 0\times16^0 + 11\times16^{-1} + 10\times16^{-2} + 11\times16^{-3}$

$= \boxed{2976.729\,248046_{10}}$

(c) $(2765.6)_8$ en base 2 :

2 7 6 5, 6

010 111 110 101. 110

$\boxed{101\ 1111\ 0101.11_2}$

(d) $(2C3.F8D)_{16}$ en base 8 :

2 C 3 . F 8 D

0010 1100 0011. 1111 1000 1101

1 3 0 3. 7 6 1 5

$\boxed{1303.7615_8}$

Question 5 (10 points)

(a) Quel est le plus **grand** entier **non signé décimal** que l'on peut représenter sur 2 octets ?

$2^{16} - 1 = \boxed{65\ 535}$

(b) Quel est le plus **petit** entier **signé décimal** que l'on peut représenter sur 16 bits ?

$\frac{-2^{16}}{2} = \boxed{-32\ 768}$

(c) Quel est le plus **grand** entier **signé décimal** que l'on peut représenter avec 3 octets ?

$\frac{2^{24}}{2} - 1 = \boxed{8\ 388\ 607}$

(d) Combien de **bits** sont nécessaires pour représenter l'entier **signé** $-123\,456_{10}$?

$\lfloor \log_2 123\ 456 \rfloor + 2 = 16 + 2$
$= \boxed{18 \text{ bits}}$

(e) Combien d'**octets** sont nécessaires pour représenter l'entier **non signé** $123\,456\,789_{10}$?

$\lfloor \log_2 123\ 456\ 789 \rfloor + 1 = 26 + 1$
$= 27$ bits

Comme 3 octets = 24 bits, il faut $\boxed{\text{4 octets}}$

Question 6 (4 points)

Pour chacune des opérations suivantes, supposons que le type *short int* (2 octets) est utilisé, en effectuant l'opération, déterminez si elles engendrent un dépassement de capacité ou non, sachant que le résultat est lui aussi mis en mémoire dans une variable de type *short int*. Justifiez.
Les nombres sont donnés en base 10.

2 octets signés : −32 768 à 32 767

(a) $3\,700 - 35\,000$ > 32 768 : dépassement

(b) $-28\,000 + 32\,000$ = 4000

ok

(c) 260×1200 = 312 000 > 37 767 : dépassement

(d) -125×125 = -15 625 ok

Question 7 (4 points)

Pour chacune des opérations suivantes, sans l'effectuer, déterminez si elles engendrent un dépassement de capacité ou non, sachant que le résultat est lui aussi mis en mémoire sur un octet. Justifiez.
Les nombres sont donnés en base 2 et sont représentés sur un octet, de façon **non signée**.

(a) $1\,1100 + 100\,0000$

5 bit 7 bits

max 7+1 = 8 bits : ok

(b) $111\,0000 + 1010\,1111$

7 bits 8 bits

max 8+1 = 9 bits : dépassement possible

(c) $111\,0000 \times 110$

7 bits 3 bits

7+3 = 10 bits : dépassement possible

(d) 1111×1101

4 bits 4 bits

4+4 = 8 bits : ok

Question 8 (6 points)

Pour chacun des nombres binaires :

- écrivez le nombre avec la notation exponentielle ;
- déterminez quel type doit être utilisé pour le représenter entre le type *float* et le type *double* et justifiez ;
- écrivez le contenu de la mantisse (23 ou 52 bits selon la réponse précédente) en arrondissant le dernier bit, s'il y a lieu ;
- déterminez si la représentation du nombre en mémoire est exacte ou non.

Rappels :

-128 à 127

- Le type *float* est sur 32 bits : 1 pour le signe, 8 pour l'exposant et 23 pour la mantisse.
- Le type *double* est sur 64 bits : 1 pour le signe, 11 pour l'exposant et 52 pour la mantisse.

-1024 à 1023

(a) $-1\,0001\,0111\,1001.\overline{1} \times 2^{-132}$

Notation exponentielle : $-1.0001\ 0111\ 1001\overline{1} \times 2^{-120}$

Type à utiliser : float , car $-120 \geq -128$

Contenu de la mantisse :

0001 0111 1001 ~~1111 1111 111~~ 1 / 1

10 0000 0000 000

La représentation du nombre en mémoire est-elle exacte ?

non, car la mantisse est infinie et on a arrondi

(b) $0.0000\,0000\,1101\,1111\,101 \times 2^{150}$

Notation exponentielle :

$1,1011\ 1111\ 01 \times 2^{141}$

Type à utiliser : double, car $141 > 127$

Contenu de la mantisse :

1011 1111 0100 0000 0000 0000 0000 0000 0000 0000 0000 0000

La représentation du nombre en mémoire est-elle exacte ?

oui, la mantisse est entièrement représentée

Question 9 (12 points)

Supposons qu'il existe un type pour représenter les nombres à virgule flottante défini comme suit :

-8 à 7

16 bits : 1 pour le signe, 4 pour l'exposant et 11 pour la mantisse.

Pour chacun des nombres binaires :

i) Écrivez-le sous la notation exponentielle ;

ii) Déterminez s'il est possible ou non de l'exprimer avec une variable de ce type. Justifiez.

iii) Si oui, donnez le contenu de la mantisse et dites s'il y a une perte de précision ou non lorsque le nombre est mis en mémoire. N'oubliez pas d'arrondir la mantisse en mémoire, s'il y a lieu.

iv) S'il y a une perte de précision, donnez la valeur décimale de la valeur mise en mémoire.

(a) $11\,1010\,1110.\overline{011}$

Notation exponentielle : $1.1101\ 0111\ \overline{011} \times 2^9$

Type approprié ? non, car $9 > 7$

Contenu de la mantisse :

La représentation du nombre en mémoire est-elle exacte ?

S'il y a une perte de précision, valeur décimale du nombre mis en mémoire :

comme le type n'est pas approprié, vous n'avez pas à repondre à ces 3 questions

(b) $110.1010\,1110\,\overline{011}_2 \times 2^3 = 53.450\,89\overline{2\,857\,14}_{10}$

Notation exponentielle : $1.1010\ 1011\ 1001\overline{1} \times 2^5$

Type approprié ? oui, car $5 \leq 7$

Contenu de la mantisse : arrondissement

$1010\ 1011\ 1001 \longrightarrow 1010\ 1011\ 101$

La représentation du nombre en mémoire est-elle exacte ?

non, on a dû arrondir

S'il y a une perte de précision, valeur décimale du nombre mis en mémoire :

$1.1010\ 1011\ 101 \times 2^5 = 110101.0111\,01_2$

$= 53,453\,15_{10}$

(c) $110.1010\,111 \times 2^{-4}$

Notation exponentielle : $1.1010\ 1011\ 1 \times 2^{-2}$

Type approprié ? oui, car $-2 \geq -8$

Contenu de la mantisse :

$1010\ 1011\ 100$

La représentation du nombre en mémoire est-elle exacte ?

oui, on n'a pas eu à arrondir

S'il y a une perte de précision, valeur décimale du nombre mis en mémoire :

X

Question 10 (6 points)

Supposons que la représentation en mémoire d'un nombre décimal est sur 16 bits, avec 1 bit pour le signe, 8 bits pour l'exposant et 7 bits pour la mantisse. Après avoir converti le nombre 12.4_{10} en binaire, l'avoir exprimé avec la notation exponentielle, déterminez la valeur décimale correspondante à ce qui est en mémoire.

(a) Conversion du nombre décimal 12.4_{10} en binaire :

12	2
6	0
3	0
1	1
0	1

2	0,4
0	0,8
1	1,6
1	1,2
0	0,4
0	0,8

$12.4_{10} = 1100.\overline{0110}_2$

(b) Notation exponentielle :

$1.1000\overline{11} \times 2^3$

(c) Contenu des 7 bits de la mantisse :

1000 110 / 0

(d) Valeur décimale de ce qui est en mémoire :

1.1000110×2^3

$= 1100.011_2$

$= \boxed{12,375_{10}}$